Chapter 7 — Practice Log

Exercises, Mistakes & Fixes

A record of the six hands-on drills worked through after Chapter 7. Each exercise notes what was attempted, where a mistake crept in, and exactly how it was diagnosed. As before, the mistakes are the most valuable part — each exposes a concept that is easy to misread from the book alone. **Blue** marks the concept, **red** the pitfall, **green** the fix.

1 Directory Creation with Brace Expansion

Task: With a single `mkdir`, create a `Photos` directory holding one sub-directory per month of 2024 and 2025, named `YEAR-MONTH` with zero-padded months (24 total), sorting chronologically.

Solved Cleanly — First Try

```
$ mkdir -p Photos/2024..2025-01..12
```

The two brace expressions **combine** — every year pairs with every month — and the `Photos/` preamble is stamped onto each of the 24 names. The shell expands the line to `mkdir -p Photos/2024-01 ... Photos/2025-12` before `mkdir` runs.

Why `-p` and Why Zero-Padding

`-p` creates the `Photos` parent as needed and stays silent if a path already exists — without it, `mkdir` would fail trying to make `Photos/2024-01` before `Photos` existed. Zero-padding matters because names sort as *text*, one character at a time: unpadded, `10` sorts right after `1` (both start with `1`), scrambling the order. Padded to equal width (`01..12`), text order and numeric order finally coincide.

2 Predict Before You Run

Task: Predict, then verify, which of `echo b*`, `echo {b,c,d}`, `echo $((10 % 3))`, `echo ~` depend on the current directory.

Three Predicted Perfectly

`{b,c,d}` → `b c d` (pure text); `$((10%3))` → `1` (pure arithmetic: remainder of $10 \div 3$); `~` → the home path (depends only on identity). All three are **directory-independent**. Only `b*` (pathname expansion) consults the filesystem.

An Unmatched Glob Prints Its Literal Pattern

The prediction for `echo b*` was "nothing." In fact, when a glob matches *no* files, the shell passes the pattern through **unchanged**:

```
$ echo b*
b*                  # literal text, not an empty line
```

Verified live in the playground. This explains a whole class of errors: `rm b*` with no matches hands `rm` the literal string `b*`, which then complains it cannot find a file named `b*`.

3 The Misspelled Variable

Task: Contrast `echo $HOME` with `echo $HOEM`.

Predicted Correctly

`$HOME` → `/home/ron`; `$HOEM` → an empty line. An unknown variable name expands to an **empty string**, silently.

Two "Misspelling" Behaviors — Side by Side

This is the real lesson, seen against Exercise 2:

- **Misspelled glob** (`b*`, no match) → literal pattern printed. *The mistake is visible.*
- **Misspelled variable** (`$HOEM`) → empty string, no error. *The mistake is invisible.*

A glob typo announces itself; a variable typo vanishes. In a script, `rm -rf $HOEM/temp` silently becomes `rm -rf /temp`. This is why scripts use `set -u` and `"${VAR}"` to make the invisible failure visible.

4 Quoting Levels on One String

Task: Run `echo ~$((3*3)) $(whoami) {x,y}` at all three quoting levels and report which expansions fire.

Rule Applied Correctly at All Three Levels

```
$ echo ~ $((3*3)) $(whoami) x,y      # none:   all four expand
/home/ron 9 ron x y
$ echo "~ $((3*3)) $(whoami) x,y"    # double: only $-forms
~ 9 ron x,y
$ echo '~ $((3*3)) $(whoami) x,y'    # single: nothing
~ $((3*3)) $(whoami) x,y
```

The "Starts with $" Shortcut

Inside double quotes, tilde and brace freeze (they do not start with `$`), while arithmetic `$((...))` and command substitution `$(whoami)` survive (they do). The complete list of survivors is the three `$`-based expansions: parameter, arithmetic, command substitution. Single quotes make no exceptions — total blackout.

5 The Command-Substitution Newline Effect

Task: Compare `echo $(ls -l)` with `echo "$(ls -l)"` — layout and argument count.

Layout Described Correctly

Unquoted → collapsed onto one line, whitespace flattened. Quoted → original multi-line layout preserved.

Argument Count Was Reversed

The counts were given backwards. The correct mapping:

```
echo $(ls -l)     -> word-splitting runs -> MANY arguments -> flattened
echo "$(ls -l)"   -> splitting suppressed -> ONE argument  -> preserved
```

The intuition that fixes the reversal: **more splitting = more arguments = more flattening**. The unquoted version splits the most, so it has the most arguments — and *that* is why it collapses onto one line: `echo` glues its many arguments back together with single spaces. The quoted version refuses to split, keeping one intact string with newlines. (The book's `df -h` example: 49 arguments unquoted vs. 1 quoted.)

Where Word-Splitting Sits

Word-splitting is **step 4** of the shell's fixed pipeline — brace, tilde, then the `$`-expansions (step 3), then *word-splitting*, then pathname expansion. It runs *on the output of* the expansions, dividing that text into arguments. This is why quoting (which disables splitting) changes the result even though the substitution ran identically both times: quoting does not stop the substitution, only the splitting that follows it.

6 Escaping

Task: Produce the literal line `Your $PATH variable and the cost is $5` using *double* quotes, with both `$` appearing literally.

Escaping Mechanics 100% Correct

```
$ echo "Your $PATH variable and the cost is $5"
Your $PATH variable and the cost is $5
```

Both `\$` did their job: the backslash strips each `$` of its special meaning, so `$PATH` did not expand to the directory list and `$5` did not vanish to empty. Inside double quotes, that is the correct way to force a literal `$`.

Minor Capitalization Typo

The submitted string read `\$Path` (lowercase) rather than `$PATH`. The escape prevents *expansion* but does not correct spelling — it prints exactly the letters typed. Worth noting: had the

backslashes been forgotten, `$PATH` would have expanded into the full colon-separated path list and `$5` would have vanished, turning the line into garbage. The two backslashes are the only thing preventing that.

Side lesson: `/n` vs. `\n`

A separate confusion surfaced mid-exercise: `echo "hello/n bye"` printed literally in both quote styles. Two reasons, both worth keeping:

Backslash vs. Forward Slash, and `-e`

- `/` (forward slash) is an **ordinary character**; `\` (backslash) is the escape character. `/n` is just two literal characters.
- Even with the right slash, `echo` interprets `\n` as a newline **only with the `-e` flag** (or inside `$'...'`).
- **Quoting** and `-e` are *independent* controls: quoting governs shell expansions and word-splitting; `-e` governs whether `echo` turns `\n` into a real newline.

```
$ echo    "a"      -> a        # no -e: literal in any quotes
$ echo -e "a"      -> a / b        # -e: real newline
```

7 Running Themes

Patterns Worth Carrying Forward

1. **Brace expansions combine** — `{a,b}-{c,d}` gives every pairing; ideal for bulk directory creation. Zero-pad numbers so they sort as text.
2. **Unmatched glob → literal pattern**; **unknown variable → empty string**. One failure is visible, the other silent.
3. **Double-quote shortcut:** the `$`-expansions survive, everything else freezes. Single quotes suppress all.
4. **More word-splitting = more arguments = more flattening.** Quote command substitution (`"$(...)"`) to preserve layout.
5. **Escaping** forces a literal character but does not fix spelling; `\` is the escape character, `/` is not.
6. **Quoting and `-e` are separate dials** — one for the shell, one for `echo`.
7. **Test on adversarial input** — data where the "easy" answer and the correct answer disagree (padded vs. unpadded, matched vs. unmatched, quoted vs. unquoted).